DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

1. What are the advantages of using Apache Spark over Hadoop MapReduce?

1. **Better Performance:** Spark is significantly faster than Hadoop MapReduce because it performs **in-memory computations**, reducing the need to read and write intermediate data to disk.
2. **Flexible Processing:** Spark supports both **batch and real-time/interactive processing**, whereas Hadoop MapReduce is primarily batch-oriented.
3. **Machine Learning Support:** Spark comes with **MLlib**, a built-in library for **iterative in-memory machine learning computations**, making ML workflows faster and more efficient.
4. **Ease of Development:** Spark provides **high-level APIs** in Python, Scala, and Java, which makes coding simpler compared to writing complex MapReduce jobs.

2. Can you explain what a DAG is in Spark and how Spark uses it to execute tasks?

In Apache Spark, a **DAG (Directed Acyclic Graph)** represents the logical sequence of operations applied to a dataset. Whenever you perform **transformations** on an RDD or DataFrame, Spark builds a DAG to keep track of these operations.

How it works:

1. **Stages:** The DAG is divided into **stages**, where each stage contains tasks that can be executed **in parallel**.
    - **Shuffle Stages:** Require **data movement (shuffling)** between nodes.
    - **Non-Shuffle Stages:** Can run without data transfer across nodes.
2. **Task Scheduling:** The **DAGScheduler** analyzes the DAG, breaks it into stages, and schedules tasks on **worker nodes** for execution efficiently.
3. **Execution:** This approach allows Spark to optimize task execution, reduce unnecessary data movement, and improve performance.

3. What are the key components of the Apache Spark ecosystem and their purposes?

1. **Spark Core:** The foundational engine for **distributed and parallel data processing**, handling task scheduling, memory management, and fault tolerance.

2. **Spark SQL:** Enables **processing of structured data** using SQL queries or the DataFrame API.
3. **Spark Streaming:** Supports **real-time data processing**, allowing continuous ingestion and analysis of streaming data.
4. **MLlib:** A library for **scalable machine learning**, providing algorithms for classification, regression, clustering, and recommendation.
5. **GraphX:** A library for **graph processing and graph-parallel computation**, useful for social network analysis, recommendations, and more.

4. Can you explain the difference between Apache Spark, Databricks, and PySpark?

1. **Apache Spark:** An **open-source big data framework** for distributed data processing and analytics. It provides the core engine for batch and streaming data processing.
2. **Databricks:** A **managed platform built on top of Spark** that offers enhanced reliability, performance, collaborative notebooks, and simplified data pipeline management for enterprises.
3. **PySpark:** A **Python API for Spark**, allowing developers to write Spark programs using **Python** instead of Scala or Java. It provides access to Spark Core, Spark SQL, MLlib, and other Spark libraries in Python.

5. Can you explain the Apache Spark architecture, specifically the roles of the Driver and Worker nodes?

1. **Driver:**
    - Acts as the **central coordinator** of the Spark application.
    - Converts user code into **tasks** and schedules them across the cluster.
    - Runs the **main program** and monitors execution progress.
2. **Workers (Executors):**
    - Run on **worker nodes** and execute tasks assigned by the driver.
    - Each executor runs in its own process and handles **task execution**, **data storage**, and **caching**.

The **driver communicates with multiple executors**, distributing tasks in parallel to process large datasets efficiently across the cluster.

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

6. What is the difference between SparkSession and SparkContext in Apache Spark?

1. **SparkContext:**
    - The **entry point** for Spark functionality in older Spark versions.
    - Responsible for **connecting to the Spark cluster**, creating **RDDs**, and coordinating **task execution**.
2. **SparkSession:**
    - Introduced in **Spark 2.0+** as a **unified entry point**.
    - Combines **SparkContext, SQLContext, and HiveContext** into a single object.
    - Used for **DataFrame and Dataset operations**, along with batch and streaming workloads.

7. Can you explain the difference between transformations and actions in Apache Spark?

1. **Transformations:**
    - Operations that **create a new RDD** from an existing one without producing a final result.
    - **Lazy evaluated** — Spark builds a **DAG** but doesn't execute them until an action is called.
    - Examples: filter(), map(), distinct(), orderBy().
2. **Actions:**
    - Operations that **trigger computation** and produce a **result** (non-RDD value) or write data to storage.
    - Examples: count(), collect(), saveAsTextFile().

8. What is the difference between narrow and wide transformations in Apache Spark, and why does it matter?

1. **Narrow Transformations:**

- Each partition of the **child RDD** depends on at most **one partition of the parent RDD**.
- Do **not require data shuffling**, making them **more efficient**.
- Examples: map(), filter(), union().

2. **Wide Transformations:**
   - A **child partition** may depend on **multiple parent partitions**, requiring **data shuffle across nodes**.
   - More **resource-intensive** due to network I/O and disk operations.
   - Examples: groupByKey(), reduceByKey(), join().

9. What is meant by lazy transformations in Apache Spark, and why are they important?

In Spark, **lazy evaluation** means that **transformations are not executed immediately** when called on an RDD or DataFrame. Instead, Spark:

1. **Records the transformations** in a **DAG (Directed Acyclic Graph)** of computation.
2. **Delays execution** until an **action** (like count() or collect()) is triggered.
3. **Optimizes execution** by combining multiple transformations and reducing unnecessary computations.

10. What strategies can be used to improve the performance of a Spark job?

Performance of a Spark job can be improved using several best practices:

1. **Use DataFrames or Datasets:** Prefer them over RDDs as they provide **built-in optimizations** like Catalyst and Tungsten.
2. **Optimize Transformations:** Use coalesce() instead of repartition() when reducing partitions to **avoid full shuffles**.
3. **Avoid UDFs when possible:** Built-in Spark functions are **more efficient** than User Defined Functions.
4. **Caching/Persisting:** Cache **intermediate results** that are reused multiple times to prevent recomputation.

5. **Reduce Shuffling:** Minimize expensive shuffle operations (groupByKey()) and prefer alternatives like reduceByKey().

11. What is the difference between a DAG and lineage in Apache Spark?

1. **DAG (Directed Acyclic Graph):**
    - Represents the **entire execution plan** of a Spark application.
    - Shows all stages and tasks Spark will execute to process data.
2. **Lineage:**
    - Tracks the **sequence of transformations** applied to a specific RDD.
    - Used for **fault recovery**, allowing Spark to recompute lost partitions from the original data.

12. Can you explain the difference between RDDs and DataFrames in Apache Spark?

1. **RDD (Resilient Distributed Dataset):**
    - A **distributed collection of objects** spread across a cluster.
    - Provides **low-level control** over data and transformations.
    - Supports **functional programming APIs** in Java, Scala, and Python.
2. **DataFrame:**
    - A **distributed collection of data organized into named columns**, similar to a **table in a relational database**.
    - Provides **high-level APIs** and built-in optimizations via the **Catalyst optimizer**.
    - Easier to work with for **SQL queries, aggregations, and analytics** compared to RDDs.

13. What is the difference between groupByKey() and reduceByKey() in Spark, and when should each be used?

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

Feature	groupByKey()	reduceByKey()
Operation	Groups **all values** for each key into a single sequence	Combines values per key using a **reduce function** (like sum, max)
Performance	Less efficient, as it **shuffles all values** across the network	More efficient, does **partial aggregation before shuffle**
Use Case	When you need the **grouped values** themselves	When you need **aggregated results**, e.g., sum, count, max

14. What is PySpark, and how does it enable Python applications to run on Spark?

- **PySpark** is the **Python API for Apache Spark**, allowing developers to write Spark applications using **Python**.
- It enables **parallel processing** of data across a distributed cluster.
- Spark itself is **written in Scala**, and PySpark uses **Py4J**, a Java library, to interface Python with the **JVM-based Spark engine**.
- To run PySpark, you need **Python, Java, and Apache Spark** installed.

15. Can you explain the Spark RDD transformations distinct(), union(), intersection(), and subtract()?

1. **distinct()**: Returns a new RDD with **duplicate elements removed**.
2. **union()**: Combines two RDDs into a **single RDD** containing all elements from both.
3. **intersection()**: Returns an RDD with **only the elements common** to both RDDs.
4. **subtract()**: Returns an RDD with **elements from the first RDD that are not present** in the second RDD.

16. What are partitions in Apache Spark, and why are they important?

- A **partition** in Spark is a **logical chunk of data** stored on a node in the cluster, representing the **unit of parallelism** for processing.
- The **number of partitions** should be chosen carefully based on the **cluster size** and **application requirements** to balance **parallelism and overhead**.

- **Common types of partitioning in Spark:**
    1. **Hash Partitioning:** Data is distributed across partitions based on a **hash function** applied to the key.
    2. **Range Partitioning:** Data is divided into **ranges**, useful for ordered data and range queries.

17. What is the difference between repartition() and coalesce() in Apache Spark, and when should each be used?

Feature	repartition()	coalesce()
Data Shuffle	Performs a **full shuffle**	Minimal or **no shuffle**
Use Case	Increase or **change the number of partitions** for better parallelism before heavy operations	**Reduce the number of partitions**, typically before writing output
Performance	Slower and more **resource-intensive**	Faster and **optimized for shrinking partitions**

18. What are the main characteristics of PySpark?

1. **Abstracted Nodes:** Individual worker nodes are **not directly addressed**; Spark handles task distribution automatically.
2. **Spark API:** Provides **high-level APIs** in Python (and other languages like Java, Scala, and R) to leverage Spark's features easily.
3. **Map-Reduce Model:** Built on the **MapReduce paradigm**, where programmers define **map and reduce functions** for data processing.
4. **Abstracted Network:** Communication between nodes is **implicit**, handled by Spark, so developers don't manage networking directly.

19. What is the difference between mapPartitions() and mapPartitionsWithIndex() in Spark?

1. **mapPartitions()**:
    - Applies a **function to each partition** of an RDD.

- Useful for **processing partition-level data efficiently** instead of element-wise.

2. **mapPartitionsWithIndex()**:
    - Similar to mapPartitions(), but also **provides the partition index** to the function.
    - Useful when you need **partition-specific logic** or debugging.

20. What is data shuffling in Spark, and why is it considered an expensive operation?

- **Data Shuffling:** The process of **redistributing data across nodes** in a cluster, typically triggered by operations like groupByKey(), reduceByKey(), and join().
- **Why it's expensive:** Shuffling involves **disk I/O, network transfer, and serialization/deserialization**, which can significantly slow down Spark jobs.
- **Optimization:** Spark reduces shuffling using **combiner functions** in transformations like reduceByKey(), which **aggregate data locally** before moving it across the network.

21. What does inferSchema do when reading a file in Spark, and when should it be used?

- **inferSchema** scans the input data and **automatically detects the data types** for each column in a DataFrame.
- By default, inferSchema is **false**, and all columns are read as **strings**. Setting it to **true** allows Spark to infer the correct types.
- **Considerations:**
    - Can **slow down data loading** for large datasets.
    - Prefer **defining a schema manually** for better performance and to avoid unexpected type issues.
    - Useful when the **input schema is dynamic** or changes frequently.

22. What is a broadcast join in PySpark, and when should it be used?

- **Broadcast Join:** When one of the DataFrames is **small enough**, Spark can **broadcast it to all worker nodes**, avoiding a full shuffle of the larger DataFrame and making the join **much faster**.
- **Example:**

```
from pyspark.sql.functions import broadcast

df1 = spark.read.csv("large_table.csv")

df2 = spark.read.csv("small_lookup.csv")

joined = df1.join(broadcast(df2), "key")
```

23. How do you register and use a User Defined Function (UDF) in PySpark?

- **UDF (User Defined Function):** Allows you to apply **custom Python functions** on PySpark DataFrames.
- **Example:**

```
from pyspark.sql.functions import udf

from pyspark.sql.types import StringType

def to_upper(s):

  return s.upper()

to_upper_udf = udf(to_upper, StringType())

df = df.withColumn("upper_name", to_upper_udf(df.name))
```

24. What are the key features of Apache Spark, and what are the advantages and disadvantages of using PySpark?

Key Features of Apache Spark:

1. **In-Memory Computing:** Processes data in RAM, leading to **faster performance** than disk-based systems.
2. **Distributed Processing:** Supports **parallel computation** across multiple nodes in a cluster.
3. **Support for Multiple Workloads:** Handles **batch, streaming, machine learning, and graph processing**.
4. **Rich APIs:** Provides APIs in **Python, Scala, Java, and R**.

5. **Fault Tolerance:** Automatically recovers lost data using **RDD lineage**.

Advantages of PySpark:

- Enables **Python developers** to leverage Spark's distributed processing.
- Supports **DataFrames, Datasets, and SQL queries**.
- Integrates with **machine learning (MLlib) and streaming**.

Disadvantages of PySpark:

- **Performance overhead** due to Python-JVM interaction via Py4J.
- UDFs in Python are **slower** than built-in Spark functions.
- **Memory management** may require tuning for large datasets.

25. What are some commonly used transformations in PySpark, and how do they work?

- **Common Transformations in PySpark:** map(), flatMap(), filter(), distinct(), union(), groupByKey(), reduceByKey().
- These transformations **create new RDDs** from existing ones and are **lazy**, meaning they are **not executed immediately**. Spark builds a **DAG of transformations**, which only runs when an **action** (like count() or collect()) is triggered.

26. Why are broadcast variables needed in Apache Spark, and how do they work?

- **Broadcast Variables:** Allow a **read-only variable** to be **cached on each worker node**, rather than sending a copy with every task.
- **Purpose:** Efficiently provide **large input datasets** to all nodes without repeated network transfer.
- Spark uses **optimized broadcast algorithms** to minimize **communication costs** and improve performance.

27. How does caching work in PySpark, and when should it be used?

- **Caching:** When you **cache an RDD or DataFrame**, Spark stores it in **memory**. This prevents **recomputing the entire transformation chain** if the data is reused later.
- **Example:**

```
df.cache()      # Mark the DataFrame to be cached
df.count()      # Triggers the actual caching
```

- **Use Case:** Ideal for **iterative algorithms** or when the same data is **accessed multiple times** in a Spark job.

28. What is the difference between repartition() and coalesce() in Spark, and when should each be used?

- Both repartition() and coalesce() are **expensive operations** because they may **shuffle data across partitions**, so they should be used carefully.

Feature	repartition()	coalesce()
Purpose	Increase or decrease the number of partitions	Only **decrease** the number of partitions
Type of Transformation	Wide transformation (causes full shuffle)	Narrow transformation (minimal or no shuffle)
Use Case	When you want **evenly distributed partitions** and need a shuffle to redistribute data	When **reducing partitions** without shuffle for optimized performance
Supported On	RDD, DataFrame, Dataset	RDD, DataFrame, Dataset

29. What is the difference between map() and flatMap() transformations in PySpark?

- **map()**: Applies a function to **each element** of an RDD and returns a **new RDD with the same number of elements** as the input.
- **flatMap()**: Applies a function to each element but can return **zero or more elements** per input element, resulting in a **flattened RDD** that may have **more or fewer elements** than the original.

30. What are the different ways to define a schema in PySpark, and why is it recommended to define one?

- **Defining a schema** allows Spark to know the **data types of columns** upfront, improving **performance and data consistency**.

Two ways to define a schema:

1. **Programmatic (using code):**
    - Define the schema using **Spark classes** like StructType and StructField.

2**. Declarative (SQL-style):**

- Define schema using a **DDL string**, similar to SQL column definitions.

31. Can you show a PySpark example that reads CSV files, joins two DataFrames, performs an aggregation, and filters rows based on a condition?

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import sum

# Create Spark session

spark = SparkSession.builder.appName("PySparkExample").getOrCreate()

# 1. Read CSV files with header

orders = spark.read.option("header", True).csv("orders.csv")

customers = spark.read.option("header", True).csv("customers.csv")

# 2. Join DataFrames on customer_id

joined_df = orders.join(customers, on="customer_id", how="inner")

# 3. Aggregate: calculate total order amount per customer

agg_df =
joined_df.groupBy("customer_id").agg(sum("order_amount").alias("total_spent"))

# 4. Filter: customers who spent more than 1000

result = agg_df.filter(agg_df.total_spent > 1000)

# Show the result

result.show()
```

32. What is the difference between cache() and persist() in PySpark?

- **cache()**:
    - Stores the DataFrame or RDD **in memory** by default.

    - Ideal for **repeated access** to the same dataset during iterative computations.

- **persist()**:
    - Allows you to **choose the storage level**, such as memory-only, memory-and-disk, or disk-only.
    - More flexible than cache(), especially for **large datasets that may not fit entirely in memory**.

33. How do you handle skewed data in Spark, and what techniques can you use to balance the load?

- **Data Skew:** Occurs when some keys have **much more data** than others, causing certain nodes to handle a disproportionate amount of work, which slows down Spark jobs.
- **Techniques to Handle Skew:**
    1. **Salting:** Add a **random suffix** to skewed keys to distribute them across partitions, then remove the suffix after processing.
    2. **Map-side combine:** Use **reduceByKey()** instead of groupByKey() to perform partial aggregation **before shuffling**.
    3. **Repartitioning:** Use **repartition() or coalesce()** to redistribute data more evenly across partitions.
    4. **Broadcast Join:** For skewed joins, broadcast the **smaller DataFrame** to all nodes to avoid shuffling large skewed keys.

34. What is Spark SQL, and how is it used for structured data processing?

- **Spark SQL** is a Spark module designed for **processing structured and semi-structured data**.
- It introduces the **DataFrame abstraction**, which represents distributed datasets with **named columns**.
- Spark SQL can act as a **distributed SQL query engine**, allowing users to **run SQL queries** on large-scale data.
- It enables **data analysts and engineers** to leverage Spark's power **without deep knowledge of Spark internals**, making transformations and queries easier.

35. How can you create a rank column in PySpark using Window functions, and what are the alternatives?

Answer (interview mode):

- **Window Functions** allow you to perform **calculations across a set of rows related to the current row**.
- **Creating a rank column:**

```
from pyspark.sql.window import Window

from pyspark.sql.functions import rank

windowSpec = Window.partitionBy("department").orderBy("salary")

df.withColumn("rank", rank().over(windowSpec)).show()
```

36. What is the difference between broadcast variables and accumulator variables in Spark?

- **Broadcast Variable:**
  - Used to **share a read-only variable** efficiently with all worker nodes.
  - Helps avoid **sending multiple copies** of large data (like lookup tables).
  - **Read-only** on the worker nodes.
  - Sent from **driver → workers**.
  - Improves **performance** by reducing network I/O.
- **Accumulator Variable:**
  - Used to **aggregate values** from workers back to the driver.
  - Common for **counters, sums, or metrics** across distributed tasks.
  - Workers can **increment**, but only the driver can read the **final value**.
  - Values flow from **workers → driver**.
  - Helps **track distributed computations** without extra storage.

37. What is the Catalyst Optimizer in PySpark, and how does it work?

- **Catalyst Optimizer** is Spark SQL's **query optimization engine**.

- It **analyzes, optimizes, and generates efficient execution plans** for DataFrame or SQL queries.
- **How it works:**
    1. **Analysis:** Resolves column names, data types, and table references in the query.
    2. **Logical Optimization:** Applies **rule-based optimizations** like constant folding, predicate pushdown, and projection pruning.
    3. **Physical Planning:** Generates multiple **physical execution plans** and chooses the **most efficient one** using cost-based optimization.
    4. **Code Generation:** Uses **whole-stage code generation** to compile query operations into optimized JVM bytecode.

38. How do you create a temporary view in PySpark, and what are its characteristics?

- **Purpose:** Temporary views allow you to **run SQL queries on DataFrames** using the Spark SQL interface.
- **Creating a Temporary View:**
- df.createOrReplaceTempView("Customer")
- spark.sql("SELECT * FROM Customer").show()
- **Characteristics:**
    - **Session-scoped:** Exists only for the current Spark session.
    - **Not persisted:** Skips saving in the underlying metastore.
    - **Dropped automatically** when the session ends.

39. What is Z-ordering in Spark Delta Lake, and how does it improve query performance?

Answer (interview mode, bullet points):

- **Definition:** Z-ordering is a technique in **Delta Lake** that **clusters data based on one or more columns** using a **Z-order curve (space-filling curve)**.
- **Purpose:** Optimizes **data skipping** during reads, improving performance for **selective queries**.

- **Use Case:** Particularly useful for queries filtering on columns like **date**, **customer_id**, or other frequently filtered attributes.
- **Example:**

```
df.write.format("delta").option("dataChange",
"false").mode("overwrite").save("/path")

spark.sql("OPTIMIZE delta.`/path` ZORDER BY (customer_id)")
```

40. What is Delta Lake, and what advantages does it offer over plain Parquet files?

- **Definition:** Delta Lake is an **open-source storage layer** on top of data lakes that adds **ACID transactions, schema enforcement, and time travel** to Parquet-based data.
- **Benefits over Parquet:**

Feature	Delta Lake	Parquet
Transactions	ACID compliant, supports updates/deletes	❌ Not ACID, only append or overwrite
Schema Enforcement / Evolution	Supported, ensures data consistency	Limited, manual handling needed
Time Travel	Supported (query historical versions)	❌ Not supported
Data Updates / Deletes	Native via MERGE or UPDATE	Requires overwrite of files
Performance Optimizations	Z-ordering, file compaction, caching	Basic storage format onl

41. What is the difference between collect(), collectAsList(), and select() in PySpark?

- **collect()**
  - Returns **all elements of an RDD or DataFrame** to the **driver** as a list of **Row objects**.
  - **Be careful** with large datasets, as it may **cause out-of-memory errors**.
- **collectAsList()**

  - Specific to **DataFrames**, returns all rows as a **Python list** (similar to collect()).
  - Slightly more **Python-friendly**, directly giving a **list of Row objects**.

- **select()**
  - Used to **choose specific columns** from a DataFrame.
  - Returns a **new DataFrame**, does **not collect data to the driver**.
  - Can be chained with transformations like filter() or groupBy().

42. How can you handle null values in PySpark DataFrames, and what are the common approaches?

- **Handling Null Values in PySpark:**
  - PySpark provides **na functions** to manage nulls in DataFrames.
- **Common Methods:**

1. **fill()** – Replace nulls with a **specified value**.
2. df.na.fill({"age": 0, "name": "Unknown"})
3. **drop()** – Remove rows containing **null values**.
4. df.na.drop()
5. **replace()** – Substitute **specific values**, including nulls.

df.na.replace({None: "N/A"})

43. How can you create a global temporary view in PySpark, and what are its characteristics?

- **Purpose:** Global temporary views allow **sharing DataFrames across multiple Spark sessions** within the same application.
- **Creating a Global View:**

df.createOrReplaceGlobalTempView("Customer")

spark.sql("SELECT * FROM global_temp.Customer").show()

- **Characteristics:**
  - Stored in a **system-preserved schema** called global_temp.

- **Session-independent:** Can be accessed from **any session** within the same Spark application.
- **Lifetime:** Remains alive until the Spark **application ends**.

44. What is Adaptive Query Execution (AQE) in Databricks Spark, and how does it improve query performance?

- **Definition:**
  - AQE allows Spark to **dynamically optimize query execution plans at runtime** based on actual data characteristics.
- **Key Features:**
  - **Dynamic join strategy selection:** Switch between **broadcast joins** and **shuffle joins** depending on table sizes.
  - **Skew join handling:** Automatically optimizes queries where some keys have **disproportionate data**.
  - **Dynamic partition pruning:** Reduces data scanned by **eliminating irrelevant partitions** during query execution.
- **Enabling AQE:**

```
spark.conf.set("spark.sql.adaptive.enabled", True)
```

45. Why are DataFrames faster than RDDs in Spark, and how do they compare with Datasets?

- **RDDs:**
  - Low-level API, operates as a collection of **Java/Scala/Python objects**.
  - Slower for operations like **grouping, aggregation, and filtering**, because it lacks **built-in optimizations**.
- **DataFrames:**
  - Higher-level abstraction with **named columns**, similar to tables in a database.
  - Leverages **Catalyst Optimizer** for query optimization and **Tungsten execution engine** for fast in-memory computation.
  - Faster for **exploratory analysis** and **aggregations on large datasets**.

- **Datasets:**
    - Combines **RDD type safety** with **DataFrame optimizations**.
    - Efficient for **strongly typed operations** and large-scale aggregations in Scala/Java

46. What is the purpose of checkpointing in Spark, and when should it be used?

- **Definition:** Checkpointing is the process of **persisting an RDD or DataFrame to a reliable storage** (e.g., HDFS) to **truncate its lineage**.
- **Purpose:**
    - Reduces the size of the **DAG** for long-running jobs.
    - **Minimizes memory overhead** by breaking long chains of transformations.
    - Prevents **recomputation** in case of **node failures**, improving job reliability.
- **Use Case:**
    - Useful for **iterative algorithms** or **jobs with very long lineage chains**.

47. How can you handle incremental loads in PySpark when a table does not have a last_modified or updated_time column?

- **Scenario:** No timestamp column exists to track changes.
- **Approach 1: Hash-Based Comparison**
    - Add a **hash column** (e.g., MD5 of concatenated columns) to each row.
    - Compare the current hash with the **previously stored hash** to identify **new or changed records**.
    - Only load **rows with changed hashes**, reducing data movement.
- **Approach 2: Full Load with Delta Merge**
    - Load the **entire dataset** and use **Delta Lake MERGE INTO** to perform **upserts**.
    - Example pseudo-code:

MERGE INTO target

USING source

ON target.id = source.id

WHEN MATCHED THEN UPDATE SET ...

WHEN NOT MATCHED THEN INSERT ...

48. How are jobs, stages, and tasks structured during a Spark job execution?

- **Job:**
    - Triggered by an **action** (e.g., collect(), show(), write()).
    - Each action corresponds to **one Spark job**.
- **Stage:**
    - Spark divides a job into **stages based on shuffle boundaries**.
    - Operations that require **data movement (like groupByKey or reduceByKey)** mark the end of a stage.
- **Task:**
    - Each stage is broken down into **tasks**, with **one task per partition**.
    - Tasks are the **smallest unit of work** executed on worker nodes.
- **Example:**

df.groupBy("col").sum().show()

- Triggers **1 job**.
- If **3 shuffles occur**, there are **3 stages**.
- If each stage has **4 partitions**, there are **4 tasks per stage**.

49. How do you perform pivot and unpivot operations on a Spark DataFrame

- **Pivot:**
    - The pivot() function **rotates data** from rows into columns.
    - Typically used with **groupBy** and an aggregation to transform **unique values of a column into separate columns**.
- Converts the month column values into **separate columns** with aggregated sales.

- **Unpivot (Melt):**
    - Transforms **columns back into rows**.
    - Useful when you want to **normalize a wide DataFrame** into a long format.
    - PySpark doesn't have a built-in unpivot function; it can be achieved using **selectExpr or stack**.

50. What are accumulators in PySpark, and in what scenarios would you use them?

- **Definition:**
    - Accumulators are **write-only shared variables** that allow **tasks to aggregate values** across a Spark job.
- **Characteristics:**
    - Tasks can **increment/update** the accumulator.
    - Only the **driver program can read** the accumulated value.
    - Cannot be used to return data to tasks.
- **Use Cases:**
    - **Counters:** Track the number of errors, missing values, or invalid records during processing.
    - **Metrics/Monitoring:** Useful for debugging and keeping track of **progress or statistics** across distributed tasks.

51. What is the difference between Spark SQL and PySpark DataFrame APIs, and when would you use each?

- **Syntax & Style:**
    - **Spark SQL:** Uses **SQL-like string queries**. Ideal for users familiar with SQL.
    - spark.sql("SELECT * FROM employees")
    - **PySpark DataFrame API:** Uses a **Pythonic, object-oriented approach** with DataFrame methods. More flexible for programmatic operations.
    - df.select("*")
- **Use Case:**

- **Spark SQL:** Quick ad-hoc queries, simple filtering, or exploratory analysis.
- **PySpark DataFrame API:** Complex transformations, chaining multiple operations, or programmatic workflows.

52. How do you filter and aggregate data in PySpark using the DataFrame API? Provide an example.

```
from pyspark.sql.functions import sum as _sum

df.groupBy("department")\

 .agg(_sum("salary").alias("sumofsalary"))\

 .filter("sumofsalary > 50000")\

 .show()
```

Explanation:

- groupBy("department") → Groups data by department.
- agg(_sum("salary")) → Aggregates salaries per group.
- alias("sumofsalary") → Renames the aggregated column.
- filter("sumofsalary > 50000") → Filters departments where the total salary exceeds 50,000.

53. How do you handle schema evolution in Azure Data Lake using Delta Lake?

- **Definition:** Schema evolution refers to **handling changes in the structure of data** (e.g., new columns, renamed columns) without breaking pipelines.
- **Handling Schema Changes in Delta Lake:**
  1. **Auto Merge Schema (Databricks):**
     - Automatically updates the table schema when writing new data with additional columns.
  2. spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")
  3. new_df.write.format("delta").mode("append").option("mergeSchema", "true").save("path")

4. **Versioning / Time Travel:**
    - Delta Lake supports **time travel**, allowing you to **revert to older schema versions** if needed.
5. **Structured Streaming with Evolving Schema:**
    - Use **schema hints** or explicitly define evolving schemas.
    - Ensures streaming jobs handle new columns safely.
6. **Monitoring Schema Drift:**
    - In **ADF Mapping Data Flows**, enable **"Allow schema drift"** to automatically handle added/removed columns.

54. Is PySpark faster than Pandas, and why would you choose PySpark over Pandas?

- **Single vs Distributed:**
    - **Pandas** runs operations on a **single machine** and a single core.
    - **PySpark** executes operations in a **distributed manner across multiple nodes**.
- **Parallel Execution:**
    - PySpark can **parallelize computations** across cores and machines, speeding up large-scale data processing.
- **Performance:**
    - For **small datasets**, Pandas may be simpler and faster due to lower overhead.
    - For **large datasets** (millions of rows), PySpark **processes operations much faster** because of **distributed computing**.

55. What is the purpose of selectExpr in PySpark DataFrames, and when would you use it?

- **Definition:**
    - selectExpr allows you to **apply SQL expressions** directly on DataFrames.
- **Purpose & Use Cases:**

- Perform **column aliasing** in a concise way.
- Execute **calculations or transformations** (e.g., arithmetic operations, string manipulations) in a **single line**.
- Convenient for **conditional transformations** using SQL-like syntax.

56. What is data skew in Spark, and why is it a problem?

- **Definition:**
  - **Skewness** (or data skew) occurs when **data is unevenly distributed across partitions**.
- **Cause:**
  - Commonly happens after **shuffle operations** like groupByKey, reduceByKey, or join.
  - Some **keys have significantly more data** than others, resulting in **large partitions**.
- **Problem:**
  - Causes **imbalanced workload**, where some tasks take much longer to execute.
  - Reduces **parallelism**, slowing down the overall Spark job.
  - Can lead to **OOM errors** if large partitions cannot fit in memory.

57. How would you troubleshoot a PySpark job that fails due to memory issues?

- **Step 1 – Monitor:**
  - Check the **Spark UI** to identify stages or tasks with **high memory usage**.
  - Look for **skewed partitions** or long-running tasks.
- **Step 2 – Optimize Transformations:**
  - Minimize **wide transformations** like groupByKey or join.
  - Use **reduceByKey or aggregateByKey** where possible to reduce shuffle data.
- **Step 3 – Tune Configurations:**

  - Adjust **executor memory**: spark.executor.memory.
  - Tune **driver memory** or **shuffle partitions** if needed.
- **Step 4 – Data Management:**
  - **Repartition** or **coalesce** data to balance partition sizes.
  - **Persist/cache** intermediate results selectively to avoid recomputation.
- **Step 5 – Additional Checks:**
  - Verify **data skew**, large joins, or nested transformations that may overload memory.

58. ***What are some common optimization techniques in Apache Spark to improve job performance?***

- **Use DataFrames/Datasets:** Prefer **DataFrames or Datasets over RDDs** as they offer **built-in optimizations** via Catalyst.
- **Optimize Transformations:**
  - Minimize **wide transformations** like groupByKey that trigger shuffles.
  - Use **reduceByKey** instead of groupByKey where possible.
- **Partitioning Strategies:**
  - Use **coalesce()** when reducing partitions to avoid unnecessary shuffles.
  - Use **repartition()** only when needed to balance large datasets.
- **Caching / Persisting:**
  - Cache or persist **intermediate results** that are reused multiple times to **avoid recomputation**.
- **Avoid UDFs When Possible:**
  - Built-in Spark functions are **more efficient** than Python UDFs.
- **Broadcast Joins:**
  - Broadcast smaller DataFrames to avoid **shuffling large datasets** in joins.
- **Adaptive Query Execution (AQE):**
  - Dynamically optimize **join strategies**, handle **skew**, and **prune partitions** at runtime.

- **File Format & Storage:**
    - Use **columnar formats** like **Parquet or Delta** for faster I/O.
    - Optimize with **Z-ordering or file compaction** in Delta Lake.

59. How would you design and implement an ETL pipeline using PySpark?

- **Step 1 – Extract:**
    - Read data from various sources such as **HDFS, Azure Data Lake, Kafka, or relational databases**.
    - Use **Spark DataFrame readers** like .read.csv(), .read.parquet(), .read.format("jdbc"), etc.
- **Step 2 – Transform:**
    - Apply **filtering, joins, aggregations, and column transformations** using PySpark DataFrame APIs.
    - Handle **data quality**: null values, duplicates, and type conversions.
    - Optimize transformations using **cache/persist**, **broadcast joins**, and **partitioning strategies**.
- **Step 3 – Load:**
    - Write the processed data to a target system such as **data lake, Delta Lake, or a data warehouse** using .write.format().save() or .write.mode("append").
- **Step 4 – Orchestration & Scheduling:**
    - Use tools like **Apache Airflow, Databricks Jobs, or Azure Data Factory** to schedule and monitor the pipeline.

60. What is salting in Spark, and how is it used to handle data skew?

- **Definition:**
    - **Salting** is a technique to **redistribute skewed keys** by adding a random "salt" column to the key, effectively **splitting large partitions into smaller ones**.
- **Purpose:**

  - Helps **avoid OOM errors** by preventing very large partitions from overloading a single task.
  - Ensures **all tasks execute**, though it does not guarantee uniform task duration.

- **How It Works:**
  - Add a **random number or string** to the skewed key.
  - Perform the operation (e.g., join or groupByKey) on the salted key.
  - After processing, **remove or merge the salt** to get the original results.
- **Trade-offs:**
  - Slightly more complex logic due to extra column.
  - May create **more smaller tasks**, increasing scheduling overhead.

61. What are the different Spark deployment modes, and how do Cluster and Client modes differ?

Feature	Cluster Mode	Client Mode
Driver location	Inside cluster	Local machine
Use case	Production	Interactive / debugging
Client connection	Can disconnect	Must stay connected
Resource management	Managed by cluster	Driver depends on client machine

62. How can you remove duplicate records from a PySpark DataFrame while keeping only the first occurrence?

- **Purpose:**
  - Remove duplicate rows while retaining only **the first occurrence**.
- **Approach 1 – Remove duplicates on all columns:**

```
deduped_df = df.dropDuplicates()

deduped_df.show()
```

- **Approach 2 – Remove duplicates based on specific columns:**

```
deduped_df = df.dropDuplicates(["column_name"])
deduped_df.show()
```

63. How would you calculate the average salary and the number of employees in each department using PySpark?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.functions import avg, count
```

- **Step 2 – Group by department and aggregate:**

```
df.groupBy("department") \
 .agg(
    avg("salary").alias("average_salary"),
    count("*").alias("employee_count")
 ) \
 .show()
```

64. Given a PySpark DataFrame of sales data, how would you identify the top 5 products based on total sales volume?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.functions import sum, col
```

- **Step 2 – Aggregate, order, and limit:**

```
top_products_df = df.groupBy("product_id") \
   .agg(sum("sales_volume").alias("total_sales")) \
   .orderBy(col("total_sales").desc()) \
   .limit(5)
top_products_df.show()
```

65. How would you filter a PySpark DataFrame to include only rows where salary is greater than 5000 and select just the name column?

- **Step 1 – Import necessary function (optional for column expressions):**

```
from pyspark.sql.functions import col
```

- **Step 2 – Filter and select:**

```
result_df = df.filter(col("salary") > 5000).select("name")

result_df.show()
```

66. How can you compute a cumulative sum of a column in PySpark for each department?

- **Step 1 – Import necessary modules:**

```
from pyspark.sql.window import Window

from pyspark.sql.functions import sum, col
```

- **Step 2 – Define the window specification:**

```
windowSpec = Window.partitionBy("department") \

        .orderBy("salary") \

        .rowsBetween(Window.unboundedPreceding, 0)
```

- **Step 3 – Compute cumulative sum:**

```
df_with_cumsum = df.withColumn("cumulative_salary",
sum("salary").over(windowSpec))

df_with_cumsum.show()
```

67. How do you perform inner, left, and right joins on two PySpark DataFrames?

- **Step 1 – Sample DataFrames:**

```
df1 = spark.createDataFrame([(1, "Alice"), (2, "Bob"), (3, "Charlie")], ["id", "name"])

df2 = spark.createDataFrame([(1, "HR"), (2, "IT"), (4, "Finance")], ["id", "department"])
```

- **Step 2 – Inner Join (common records only):**

```
inner_df = df1.join(df2, on="id", how="inner")

inner_df.show()
```

- **Step 3 – Left Join (all records from left, matched from right):**

```
left_df = df1.join(df2, on="id", how="left")

left_df.show()
```

- **Step 4 – Right Join (all records from right, matched from left):**

```
right_df = df1.join(df2, on="id", how="right")
right_df.show()
```

68. How would you find employees in PySpark who joined within the last six months?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.functions import current_date, date_sub
```

- **Step 2 – Filter for recent joiners:**

```
recent_joins_df = df.filter(df["join_date"] >= date_sub(current_date(), 180))
recent_joins_df.show()
```

69. How would you perform a right join in PySpark to get all customers, including those who haven't placed any orders?

- **Step 1 – Sample DataFrames:**

```
customers = spark.createDataFrame(
  [(1, "Alice"), (2, "Bob"), (3, "Charlie")], ["customer_id", "name"]
)
orders = spark.createDataFrame(
  [(1, 100), (1, 200), (2, 150)], ["customer_id", "order_amount"]
)
```

- **Step 2 – Perform Right Join:**

```
from pyspark.sql.functions import col
right_join_df = orders.join(customers, on="customer_id", how="right")
right_join_df.show()
```

70. How do you join two PySpark DataFrames using multiple columns as the join condition?

- **Step 1 – Use multiple columns in the join condition:**

```
joined_df = df1.join(
   df2,
   (df1["column1"] == df2["column1"]) & (df1["column2"] == df2["column2"]),
   "inner"
)
joined_df.show()
```

71. How can you rename columns in a PySpark DataFrame?

- **Method 1 – Using withColumnRenamed() for a single column:**

```
df_renamed = df.withColumnRenamed("old_column_name", "new_column_name")
df_renamed.show()
```

- **Method 2 – Renaming multiple columns using toDF():**

```
df_renamed = df.toDF("new_col1", "new_col2", "new_col3")
df_renamed.show()
```

72. How would you calculate the salary difference between consecutive employees within each department in PySpark?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.window import Window
from pyspark.sql.functions import lag, col
```

- **Step 2 – Define a window specification:**

```
windowSpec = Window.partitionBy("department").orderBy("salary")
```

- **Step 3 – Calculate salary difference using lag():**

```
salary_diff_df = df.withColumn(
   "salary_diff",
   col("salary") - lag("salary", 1).over(windowSpec)
)
salary_diff_df.show()
```

73. How can you sort a PySpark DataFrame based on one or more columns?

- **Step 1 – Import necessary functions (optional for descending order):**

```
from pyspark.sql.functions import col
```

- **Step 2 – Sort by a single column (ascending by default):**

```
df_sorted = df.orderBy("column_name")
df_sorted.show()
```

- **Step 3 – Sort by multiple columns with different orders:**

```
df_sorted = df.orderBy(col("col1").asc(), col("col2").desc())
df_sorted.show()
```

74. If a DataFrame column contains comma-separated values, how can you split it into multiple individual columns in PySpark?

- **Step 1 – Import the required function:**

```
from pyspark.sql.functions import split, col
```

- **Step 2 – Split the column into an array:**

```
df_split = df.withColumn("split_col", split(df["comma_separated_column"], ","))
```

- **Step 3 – Extract array elements into separate columns:**

```
df_split = df_split.select(
  col("split_col").getItem(0).alias("col1"),
  col("split_col").getItem(1).alias("col2"),
  col("split_col").getItem(2).alias("col3")
)
df_split.show()
```

75. How can you rank employees within each department based on their salary using PySpark?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.window import Window

from pyspark.sql.functions import rank, col
```

- **Step 2 – Define the window specification:**

```
windowSpec = Window.partitionBy("department").orderBy(col("salary").desc())
```

- **Step 3 – Apply the rank() function:**

```
ranked_df = df.withColumn("salary_rank", rank().over(windowSpec))

ranked_df.show()
```

76. How can you calculate a running total (cumulative sum) of a column for each category in a PySpark DataFrame?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.window import Window

from pyspark.sql.functions import sum, col
```

- **Step 2 – Define the window specification:**

```
windowSpec = Window.partitionBy("category").orderBy("date").rowsBetween(Window.unboundedPreceding, 0)
```

- **Step 3 – Calculate the running total:**

```
running_total_df = df.withColumn("running_total", sum("sales").over(windowSpec))

running_total_df.show()
```

77. How can you reduce the number of partitions in a PySpark DataFrame using coalesce()?

- **Purpose:**
    - coalesce() is used to **reduce the number of partitions** without performing a full shuffle.
    - It is optimized for **shrinking partitions** and improving write performance when writing small files.
- **Example Usage:**

```
# Reduce DataFrame partitions to 2
df_reduced = df.coalesce(2)
print(df_reduced.rdd.getNumPartitions())
```

78. How can you retrieve the nth highest salary from a PySpark DataFrame?

- **Step 1 – Import necessary functions:**

```
from pyspark.sql.window import Window
from pyspark.sql.functions import dense_rank, col
```

- **Step 2 – Define the window specification:**

```
windowSpec = Window.orderBy(col("salary").desc())
```

- **Step 3 – Apply dense_rank() and filter:**

```
n = 3  # nth highest salary
nth_salary_df = df.withColumn("rank", dense_rank().over(windowSpec)) \
        .filter(col("rank") == n)
nth_salary_df.show()
```

79. What are the different methods to read data into a PySpark DataFrame?

- **1. Reading CSV Files:**

```
df = spark.read.option("header", True).csv("path/to/file.csv")
```

- Options like header=True, inferSchema=True can be used.
- **2. Reading JSON Files:**

```
df = spark.read.json("path/to/file.json")
```

- **3. Reading Parquet Files:**

```
df = spark.read.parquet("path/to/file.parquet")
```

- Optimized for Spark and supports schema evolution.
- **4. Reading Delta Tables:**

```
df = spark.read.format("delta").load("path/to/delta_table")
```

- Supports ACID transactions, time travel, and schema evolution.

- **5. Reading from JDBC/Databases:**

```
df = spark.read.format("jdbc") \
    .option("url", "jdbc:mysql://host:port/db") \
    .option("dbtable", "table_name") \
    .option("user", "username") \
    .option("password", "password") \
    .load()
```

- **6. Reading from Hive Tables:**

```
df = spark.table("database_name.table_name")
```

- **7. Reading from Streaming Sources (Structured Streaming):**

```
df = spark.readStream.format("kafka").option("kafka.bootstrap.servers", "host:port").load()
```

80. How can you pivot a PySpark DataFrame based on a category column and aggregate values?

- **Step 1 – Import functions if needed:**

```
from pyspark.sql.functions import sum, col
```

- **Step 2 – Use groupBy() and pivot():**

```
pivoted_df = df.groupBy("id") \
        .pivot("category") \
        .agg(sum("value").alias("total_value"))
```

- **Step 3 – View the result:**

```
pivoted_df.show()
```

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63

DHANASEKARAN D
www.linkedin.com/in/dhanasekarand63